

University Of Asia Pacific

Department of Computer Science and Engineering

CSE 402: Operating System Lab

LAB REPORT

Report Number: 2
Report Title: SJF | Comparison with FCFS

Submitted by:
Name: Md. Muhaimin Al Baki Raiyan
Student ID: 23101163
Section: C2

Submitted To:
Atia Rahman Orthi
Lecturer,
Department of Computer Science and Engineering

Date of Submission: 10.08.2026

Comparative Study of FCFS and SJF (Non-Preemptive) CPU Scheduling Algorithms

1. Objective

The objective of this experiment is to implement and compare two non-preemptive CPU scheduling algorithms — First Come First Served (FCFS) and Shortest Job First (SJF) — using a Python program. The program computes the Completion Time (CT), Turnaround Time (TAT), and Waiting Time (WT) for a given set of processes, generates a Gantt chart for each algorithm, and evaluates which algorithm performs better in terms of average TAT and average WT.

2. Theory

2.1 First Come First Served (FCFS)

FCFS is the simplest CPU scheduling algorithm. Processes are executed strictly in the order they arrive in the ready queue, with no regard to their burst time. Once a process starts execution, it runs to completion (non-preemptive). While easy to implement, FCFS can cause the convoy effect, where short processes get stuck waiting behind a long process.

2.2 Shortest Job First (SJF) - Non-Preemptive

SJF selects, from among the processes that have already arrived, the one with the smallest burst time to execute next. Once selected, the process runs to completion without interruption. SJF minimizes average waiting time among non-preemptive algorithms, but it can cause starvation for longer processes if shorter ones keep arriving.

3. Input Data

The following process set (arrival time and burst time in milliseconds) was used for both algorithms:

PID	AT	BT	CT	TAT	WT
P1	3	3	-	-	-
P2	2	5	-	-	-
P3	5	4	-	-	-
P4	1	3	-	-	-
P5	6	2	-	-	-

4. Source Code

The algorithms were implemented in Python (PyCharm) as shown below:

```
# CSE 402 - OS Lab
# FCFS vs SJF (Non-Preemptive) CPU Scheduling Comparison

processes = [
    {'pid': 'P1', 'at': 3, 'bt': 3},
    {'pid': 'P2', 'at': 2, 'bt': 5},
    {'pid': 'P3', 'at': 5, 'bt': 4},
    {'pid': 'P4', 'at': 1, 'bt': 3},
```

```
    {'pid': 'P5', 'at': 6, 'bt': 2},
]


def fcfs(processes):
    remaining = sorted(processes, key=lambda p: p['at'])
    completed = []
    time = 0
    gantt = []

    for p in remaining:
        if p['at'] > time:
            gantt.append(('IDLE', time, p['at']))
            time = p['at']

        p = dict(p)
        start = time
        finish = time + p['bt']
        gantt.append((p['pid'], start, finish))

        p['ct'] = finish
        p['tat'] = finish - p['at']
        p['wt'] = p['tat'] - p['bt']
        completed.append(p)

        time = finish

    return completed, gantt


def sjf_non_preemptive(processes):
    remaining = [dict(p) for p in sorted(processes, key=lambda p: p['at'])]
    completed = []
    time = 0
    gantt = []

    while remaining:
        available = [p for p in remaining if p['at'] <= time]

        if not available:
            next_time = min(p['at'] for p in remaining)
            gantt.append(('IDLE', time, next_time))
            time = next_time
            continue

        current = min(available, key=lambda p: (p['bt'], p['at']))
        remaining.remove(current)

        start = time
        finish = time + current['bt']
        gantt.append((current['pid'], start, finish))

        current['ct'] = finish
        current['tat'] = finish - current['at']
        current['wt'] = current['tat'] - current['bt']
        completed.append(current)

        time = finish

    return completed, gantt
```

```
def averages(completed):
    n = len(completed)
    avg_tat = sum(p['tat'] for p in completed) / n
    avg_wt = sum(p['wt'] for p in completed) / n
    return avg_tat, avg_wt


def print_results(name, completed, gantt):
    print(f" {name} ")
    print("Gantt Chart:")
    print(" | ".join(f"{pid}({s}-{f})" for pid, s, f in gantt))
    print()
    print(f"{'PID':<5}{'AT':<5}{'BT':<5}{'CT':<5}{'TAT':<5}{'WT':<5}")
    for p in sorted(completed, key=lambda x: x['pid']):
        print(f"{p['pid']:<5}{p['at']:<5}{p['bt']:<5}{p['ct']:<5}{p['tat']:<5}
{p['wt']:<5}")

    avg_tat, avg_wt = averages(completed)
    print(f"\nAverage TAT: {avg_tat:.2f}")
    print(f"Average WT: {avg_wt:.2f}\n")
    return avg_tat, avg_wt


fcfs_completed, fcfs_gantt = fcfs(processes)
sjf_completed, sjf_gantt = sjf_non_preemptive(processes)

fcfs_tat, fcfs_wt = print_results("FCFS", fcfs_completed, fcfs_gantt)
sjf_tat, sjf_wt = print_results("SJF (Non-Preemptive)", sjf_completed, sjf_gantt)

print("Comparison")
print(f"{'Algorithm':<10}{'Avg TAT':<12}{'Avg WT':<12}")
print(f"{'FCFS':<10}{fcfs_tat:<12.2f}{fcfs_wt:<12.2f}")
print(f"{'SJF':<10}{sjf_tat:<12.2f}{sjf_wt:<12.2f}")

if sjf_tat < fcfs_tat:
      print(f"\nSJF gives lower average TAT (better by {fcfs_tat - sjf_tat:.2f})")
elif fcfs_tat < sjf_tat:
     print(f"\nFCFS gives lower average TAT (better by {sjf_tat - fcfs_tat:.2f})")

if sjf_wt < fcfs_wt:
       print(f"SJF gives lower average WT (better by {fcfs_wt - sjf_wt:.2f})")
elif fcfs_wt < sjf_wt:
    print(f"FCFS gives lower average WT (better by {sjf_wt - fcfs_wt:.2f})")
```

5. Results

5.1 FCFS Scheduling

Gantt Chart: IDLE(0-1) | P4(1-4) | P2(4-9) | P1(9-12) | P3(12-16) | P5(16-18)

PID	AT	BT	CT	TAT	WT
P1	3	3	12	9	6
P2	2	5	9	7	2
P3	5	4	16	11	7
P4	1	3	4	3	0

P5	6	2	18	12	10

Average Turnaround Time (TAT) = 8.40 ms

Average Waiting Time (WT) = 5.00 ms

5.2 SJF (Non-Preemptive) Scheduling

`Gantt Chart:  IDLE(0-1) | P4(1-4) | P1(4-7) | P5(7-9) | P3(9-13) | P2(13-18)`

PID	AT	BT	CT	TAT	WT
P1	3	3	7	4	1
P2	2	5	18	16	11
P3	5	4	13	8	4
P4	1	3	4	3	0
P5	6	2	9	3	1

Average Turnaround Time (TAT) = 6.80 ms

Average Waiting Time (WT) = 3.40 ms

6. Comparison

Algorithm	Avg. TAT (ms)	Avg. WT (ms)
FCFS	8.40	5.00
SJF	6.80	3.40

SJF gives a lower average TAT, better by 1.60 ms compared to FCFS.

SJF gives a lower average WT, better by 1.60 ms compared to FCFS.

7. Discussion

- FCFS executes processes strictly by arrival order. In this dataset, P2 (burst time 5) runs early and delays P1, P3, and P5, which increases the overall average waiting time — a clear example of the convoy effect.
- SJF instead picks the shortest available job at each decision point. This lets short processes such as P4, P1, and P5 finish quickly, pulling the average TAT and WT down significantly.
- The trade-off is visible in P2's waiting time under SJF (WT = 11 ms), which is worse than under FCFS (WT = 2 ms). Because shorter jobs keep being prioritized over it, P2 is repeatedly pushed back — this is the classic starvation risk associated with SJF.
- Both algorithms produced identical results for P4, since it was the only process ready at time 1 and had the shortest burst time regardless of the algorithm used.
- An idle period (0-1 ms) appears in both Gantt charts because no process had arrived yet at time 0; the CPU sits idle until P4 arrives at t = 1.
- Execution order differs sharply between the two algorithms after the initial idle slot. FCFS locks in the order P4, P2, P1, P3, P5 purely based on arrival, while SJF reorders the same processes to P4, P1, P5, P3, P2 based on burst time. This reordering is exactly what produces the improvement in average metrics, since it front-loads short jobs instead of letting a long job (P2) block the queue early on.

- Looking at individual process behaviour rather than just averages: P1 improves from WT = 6 ms (FCFS) to WT = 1 ms (SJF), and P5 improves from WT = 10 ms to WT = 1 ms. These are the two biggest beneficiaries of SJF, since both have short burst times (3 ms and 2 ms) but arrive relatively late, meaning FCFS would otherwise force them to wait behind everything that arrived earlier.
- P3 also improves under SJF (WT drops from 7 ms to 4 ms), though less dramatically, since its burst time (4 ms) is only moderately short relative to the others.
- The only process that gets worse under SJF is P2, and this single regression is large enough (WT rises from 2 ms to 11 ms) that it illustrates why SJF, despite being provably optimal for average waiting time, is not necessarily 'fair' at the level of individual processes — a scheduler optimizing for the mean can still treat specific jobs poorly.
- This experiment also highlights a practical limitation of SJF: it requires the burst time of each process to be known in advance. In FCFS, no such information is needed, which makes it easier to implement in a real operating system where future CPU burst lengths are generally not known ahead of time and must be estimated (e.g. using exponential averaging, as in the SJF-prediction / SRTF variants studied in the extended syllabus).
- If the workload changes — for example, if P2 had arrived after all the short jobs instead of second — the gap between FCFS and SJF would shrink, since FCFS would then happen to run short jobs early anyway. This shows that SJF's advantage is workload-dependent: it is largest precisely when a long process arrives early and would otherwise block several short ones, as happened here with P2.
- Neither algorithm is preemptive in this experiment, so once a process starts running it cannot be interrupted even if a shorter process arrives during its execution. A preemptive variant of SJF (Shortest Remaining Time First, SRTF) would very likely improve on these results further, since it could interrupt P2 the moment a shorter process such as P1 or P5 arrives, rather than waiting for P2 to finish first.

8. Conclusion

For the given process set, SJF (Non-Preemptive) outperforms FCFS on both average turnaround time and average waiting time, confirming the theoretical expectation that SJF is optimal among non-preemptive algorithms for minimizing average waiting time. However, FCFS remains simpler to implement and guarantees fairness in the sense that no process is ever overtaken once it begins waiting, whereas SJF can starve longer processes such as P2 in this experiment. The choice between the two therefore depends on whether the priority is overall system throughput (favoring SJF) or predictability and fairness of execution order (favoring FCFS).